



SERVICES & NETWORKING

Book 03 of the series — from NodePort demo tricks to production traffic:
LoadBalancer, Ingress & TLS termination, the controller ecosystem, NetworkPolicy
zero-trust, CoreDNS internals, and the Gateway API that comes next.

 **LoadBalancer**
cloud entry point

 **Ingress**
host & path routing

 **TLS**
terminate at the edge

 **Controllers**
nginx · traefik...

 **NetworkPolicy**
default-deny

 **CoreDNS**
service discovery

 **Gateway API**
the successor

 **4 Labs**
prove it hands-on

PART I · BEYOND NODEPORT

- ☐ **CH 01** The Exposition Ladder p.3
- ☐ **CH 02** LoadBalancer — The Cloud Type p.4
- ☐ **CH 03** The Problem Ingress Solves p.5

PART II · INGRESS

- ☐ **CH 04** Ingress & IngressClass p.6
- ☐ **CH 05** Host & Path Routing p.7
- ☐ **CH 06** TLS Termination p.8
- ☐ **CH 07** Ingress Controllers p.9

PART III · NETWORKPOLICY

- ☐ **CH 08** The Default-Allow World p.10
- ☐ **CH 09** NetworkPolicy — Ingress Rules p.11
- ☐ **CH 10** Egress + The Default-Deny Playbook p.12

PART IV · DNS

- ☐ **CH 11** CoreDNS — Discovery Internals p.13
- ☐ **CH 12** dnsPolicy & DNS Debugging p.14

PART V · WHAT'S NEXT

- ☐ **CH 13** Gateway API — The Successor p.15

PART VI · LABS

- ☐ **LAB 1** Path Routing via Ingress p.16
- ☐ **LAB 2** TLS on an Ingress p.17
- ☐ **LAB 3** Default-Deny → Selective Allow p.18
- ☐ **LAB 4** DNS Debugging from Inside a Pod p.19

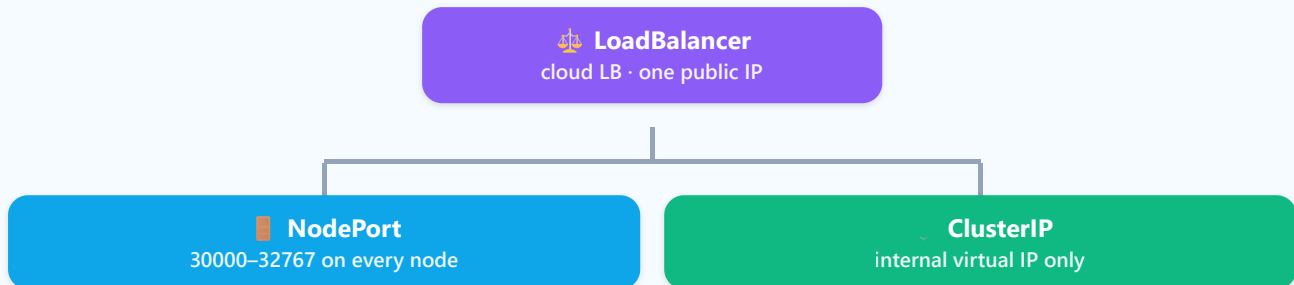
PART VII · PROVE IT

- ☐ **QUIZ** Self-Check Quiz p.20
- ☐ **KEY** Answer Key + Scoring p.21
- ☐ **GLOSS** Glossary p.22
- ☐ **CHEAT** One-Page Cheat Sheet p.23
- ☐ **FINAL** Where to Next → Book 04 p.24



Assumes Books 01–02. You know ClusterIP/NodePort and the virtual-IP trick (Book 01 · CH 25–28), service DNS basics (Book 01 · CH 12), and Secrets — including the `tls` type (Book 02 · CH 05–07). This book takes traffic from "port 30080 and pray" to production-grade entry and zero-trust inside.

Three ways to reach a Service, each one delegating to the one below it. Book 01 showed the first two — this chapter frames them as a **ladder**, because every rung exists to fix the rung beneath.

THE LADDER — TOP IS FOR HUMANS, BOTTOM IS MACHINERY

LB forwards to NodePorts · NodePorts forward through kube-proxy DNAT to Pods — each rung hides the one below

Rung	Reach	Kills it
ClusterIP	inside the cluster only	literally unreachable from outside
NodePort	any node's IP + a high port	raw ports, users hate <code>:30080</code> , no LB across nodes, no TLS story
LoadBalancer	one cloud IP per Service	\$\$\$ per Service — 10 services = 10 LB bills, and still no path routing



One sentence: the ladder is a delegation chain — *LoadBalancer* → *NodePort* → *ClusterIP* → *Pods*. NodePort solved reachability but not usability; LoadBalancer solved usability but not your cloud invoice. The next rung up — **Ingress** — fixes the invoice.

First, the cloud rung in detail — [CH 02: LoadBalancer](#)

[↑ back to top](#)

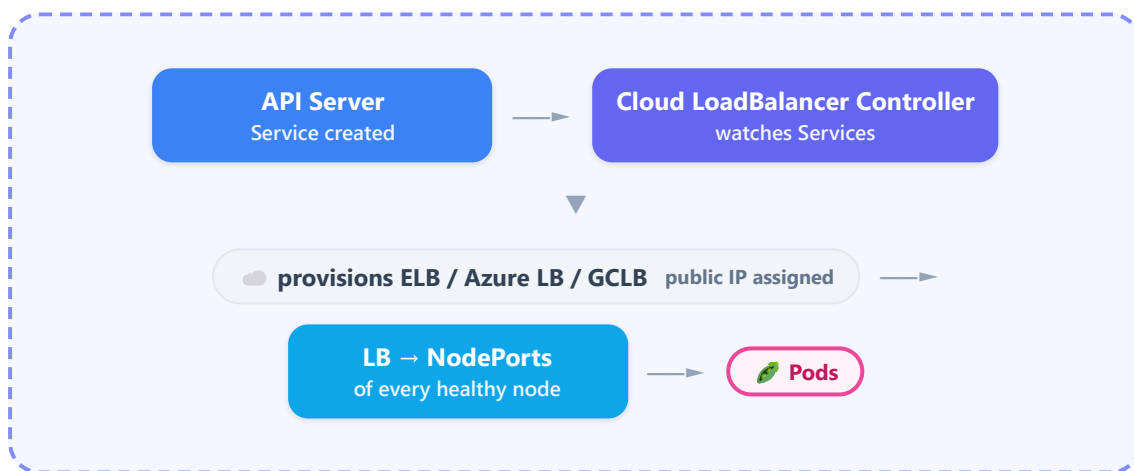


```

apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: LoadBalancer    # on a supported platform
  selector: { app: web }
  ports:
    - port: 80
      targetPort: 8080

```

🔗 What actually happens on apply



```

$ kubectl get svc web
NAME      TYPE          EXTERNAL-IP   PORT(S)          #
web       LoadBalancer <pending>     80:31234/TCP     # provisioning...
web       LoadBalancer 203.0.113.7   80:31234/TCP     # ready ✓

```

🔍 The NodePort in the port column is the tell: **LoadBalancer is NodePort underneath** — the LB just fronts it

🏠 **No cloud?** On `minikube` and `minikube tunnel` the `<pending>` becomes a real local IP; **MetalLB** to play the cloud controller's role

💰 **Cost model:** one cloud LB *per Service* that asks — this is exactly the tax CH 03 abolishes



`<pending>` **forever?** Either your platform has no LB controller (plain kubeadm, kind, Docker Desktop) or the cloud quota/IAM is blocking. The Service is never "broken" — the environment just can't fulfill the request.



One sentence: *LoadBalancer = NodePort + a cloud wallet.* Great for the one true edge of an app; terrible as a per-microservice habit.

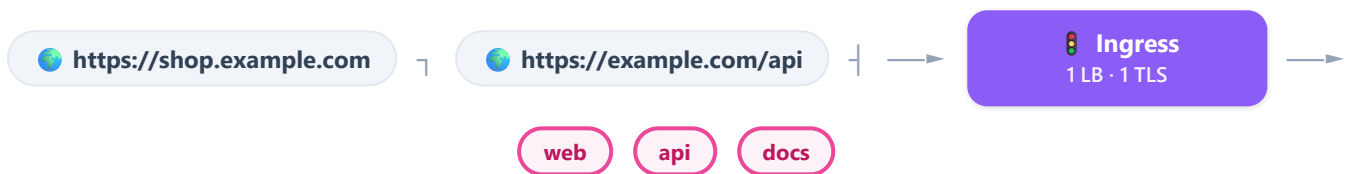
**✗ NAIVE: ONE LB PER SERVICE**

```
cloud LB #1 → web-svc
cloud LB #2 → api-svc
cloud LB #3 → admin-svc
cloud LB #4 → docs-svc
$ × 4 · TLS certs × 4 · WAF × 4
· no shared path rules · bill pain
```

✓ INGRESS: ONE EDGE, MANY ROUTES

```
cloud LB (just 1)
└─ Ingress controller
   ├── shop.example.com      → web-svc
   ├── example.com/api       → api-svc
   ├── example.com/admin     → admin-svc
   └─ docs.example.com       → docs-svc
1 bill · 1 TLS cert · L7 rules as YAML
```

Ingress is **Layer-7** awareness at the edge: it reads the HTTP **hostname** and **path** and routes accordingly — something NodePort/LoadBalancer (dumb TCP forwards) fundamentally cannot do.



Apartment update: LoadBalancer built every visitor their own private bridge to one apartment. Ingress hired a **front desk**: one entrance, a directory, and it reads the address on your letter to send you to the right apartment.



One sentence: Ingress consolidates N cloud load balancers into one, and upgrades dumb port-forwarding into **hostname/path-aware HTTP routing** — declared as plain YAML, versioned with your app.

↑ back to top



Two facts before any YAML: **(1)** an Ingress is just a rulebook — **nothing forwards traffic until a controller reads it**; **(2)** `ingressClassName` declares *which* controller owns it.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: main
spec:
  ingressClassName: nginx      # which referee handles this rulebook
  rules:
  - host: demo.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service: { name: web, port: { number: 80 } }
```

Piece	Role	Apartment translation
Ingress	the routing rulebook (declarative YAML)	the front-desk directory
IngressClass	named type of controller; routes rulebooks to referees	which security firm staffs the desk
Ingress Controller	actual reverse proxy watching Ingress objects	the desk itself



The silent trap: `kubectl apply` an Ingress with **no controller installed** and you get **zero errors** — the object is stored, accepted, and does nothing. The classic symptom: Ingress exists, `get ingress` shows no ADDRESS, curl hangs. Check `kubectl get ingressclass` first.

```
# minikube – one command installs ingress-nginx + the "nginx" IngressClass:
$ minikube addons enable ingress
$ kubectl get pods -n ingress-nginx
ingress-nginx-controller-xxxxx 1/1 Running
```



Rulebook vs referee: the Ingress *object* states intent; the *controller* is the running reverse proxy that enforces it. Kubernetes ships the grammar — you supply the proxy.

↑ back to top



The full routing surface: multiple **hosts**, each with multiple **paths**, each pointing at its own Service.

```
spec:
  ingressClassName: nginx
  rules:
  - host: shop.example.com           # name-based virtual host
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend: { service: { name: api-svc, port: { number: 80 } } }
      - path: /
        pathType: Prefix
        backend: { service: { name: web-svc, port: { number: 80 } } }
  - host: docs.example.com          # second host, same edge
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: { service: { name: docs-svc, port: { number: 80 } } }
```

pathType	Matches	/api/v1?
Prefix	path <i>and everything under it</i> , by segments	✓ yes (for /api)
Exact	only the literal path	✗ no — only /api itself
ImplementationSpecific	controller decides (nginx treats it like Prefix + regex support)	controller-defined

Order doesn't matter — nginx sorts by longest-prefix, so /api wins over / no matter where it's listed

No host field at all = catch-all **every** hostname hitting the edge (handy, but easy to shadow specific rules — be deliberate)



Testing locally: map the hostname in /etc/hosts → <minikube-ip> demo.local shop.example.com, then plain `curl http://shop.example.com/api`. No DNS purchased — the Host header does the routing.



Mental model: hosts are **apartments in the directory**, paths are **rooms within them**. One edge serves the whole building — the request's Host header is the mailman reading the label.

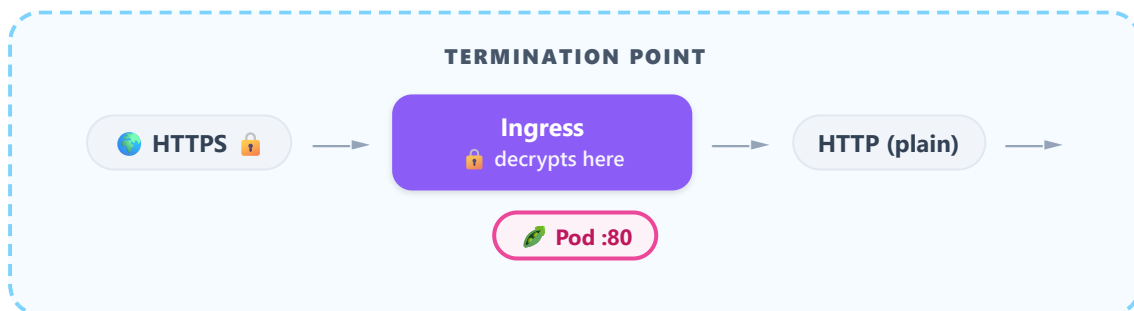
↑ back to top



"Termination" = the TLS handshake **ends at the ingress**. Users get HTTPS; your Pods receive plain HTTP on the internal network. One cert, managed once, at the edge.

```
spec:
  tls:
  - hosts: [shop.example.com]
    secretName: shop-tls      # type kubernetes.io/tls (Book 02 · CH 05)
  rules:
  - host: shop.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: { service: { name: web-svc, port: { number: 80 } } } # plain HTTP here!
```

```
# the secret the edge serves (self-signed for labs – Lab 2 does it live):
$ openssl req -x509 -nodes -newkey rsa:2048 -days 365 \
  -keyout tls.key -out tls.crt -subj "/CN=shop.example.com"
$ kubectl create secret tls shop-tls --cert=tls.crt --key=tls.key
```



Re-encryption (HTTPS all the way to Pods) is possible but controller-specific — check before promising "end-to-end encrypted" in a security review

Production certs come from cert-manager (Let's Encrypt automation) — `secretName` mechanism, the cert just appears

Missing/expired secret named in `tls` → nginx serves a **fake default cert** ; browsers scream. Diagnose: check the cert issuer, not just the padlock



One sentence: TLS terminates at the edge — the padlock in the user's bar is the Ingress's certificate; inside the cluster, traffic rides the private network as HTTP unless you deliberately re-encrypt.

↑ back to top



A controller is a **reverse proxy in a Pod** + a reconciliation loop: watch Ingress objects → render proxy config → hot-reload. The `ingress-nginx` controller literally writes an `nginx.conf` for your rulebook.

```

Ingress objects (the rulebooks)
  | watch
  ▼
Controller Pod render config → reload proxy
  |
  ▼
nginx/traefik/haproxy the process actually serving :80/:443
  
```

Controller	Underlying proxy	Know it for
ingress-nginx	nginx	the de-facto standard; huge annotation ecosystem; minikube addon
Traefik	traefik (Go)	CERT-by-default ACME, CRD-native, popular in homelab/k3s
HAProxy	haproxy	raw TCP performance pedigree
AWS ALB / GCE	managed cloud LB	Ingress provisions <i>real</i> ALB/GLB instead of in-cluster proxy



Annotation culture — power and trap: behavior beyond core routing (rewrites, CORS, IP allowlists, timeouts) arrives via per-controller **annotations**, e.g. `nginx.ingress.kubernetes.io/rewrite-target: /`. Powerful — but **non-portable**: swap controllers and those annotations are silently meaningless. K8s-native versions of some of this live in the Gateway API (CH 13).



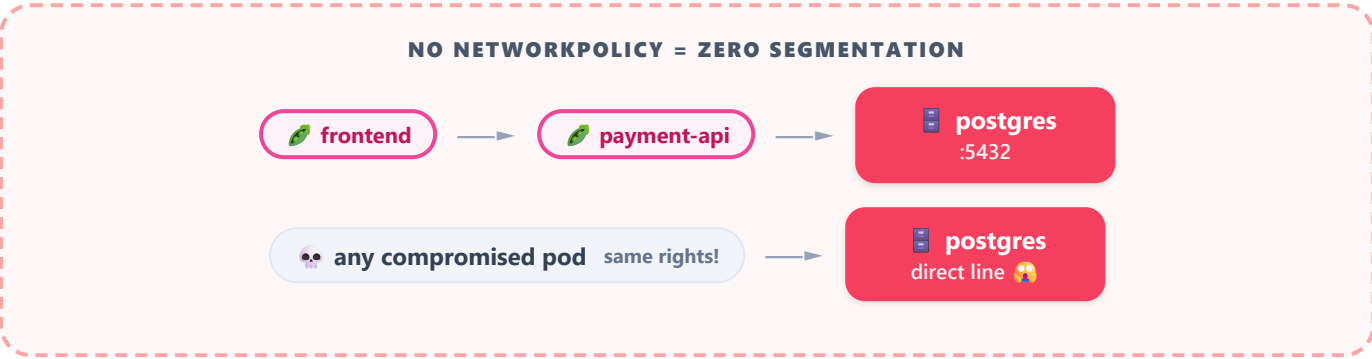
Which nginx is which? `ingress-nginx` = community/kubernetes project. `nginx-ingress` = F5/NGINX Inc's separate product. Same proxy core, different projects, different annotations — a classic interview gotcha.



One sentence: the controller is the muscle behind the rulebook — pick one, pin its `IngressClass`, and treat its annotations as vendor glue to be minimized.

↑ back to top

Book 01 built the flat network: every Pod gets a routable IP and — **by default** — **may talk to every other Pod**. Frontend to database, and *your-laptop-sidecar to database*, all equally welcome.



Assumption	Reality
"Only my Services can reach my Pods"	✗ Services are for <i>discovery</i> — Pod-to-Pod by raw IP works regardless
"Namespaces isolate the network"	✗ Book 01 · CH 11: namespaces partition names , not traffic — cross-namespace Pod calls flow freely by default
"We'd notice lateral movement"	✗ nothing logs what was never segmented

 **Why flat-by-default?** The IP-per-Pod model promises "Pods are like hosts on one big network" — zero-config apps. The price is that **security becomes opt-in**, via the object the next chapter teaches.

 **One sentence:** the Kubernetes network starts as a single open room — **any pod, any pod, forever** — until a NetworkPolicy walls off a table. Zero-trust here is a YAML you have to write.

[↑ back to top](#)



The grammar in one sentence: "Pods selected by `podSelector` may receive connections only from allowed sources, only on allowed ports." The instant one policy selects a Pod, default-allow flips to default-deny *for that Pod*.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-api
  namespace: shop
spec:
  podSelector:           # THE TARGET — who is being protected
    matchLabels: { app: api }
  policyTypes: [Ingress]
  ingress:
    - from:              # allowed SOURCES (any of these)
      - podSelector:     # same-namespace pods with...
        matchLabels: { app: frontend }
      - namespaceSelector: # ...OR all pods from matching namespaces
        matchLabels: { kubernetes.io/metadata.name: monitoring }
    ports:
      - { protocol: TCP, port: 8080 } # ...only this port
```



The AND/OR trap (interview favorite): inside **one** `from` list item, `podSelector` + `namespaceSelector` = **AND** (pods matching this label *in those* namespaces). Across **separate** items = **OR** (either source). One character of YAML indentation changes the security meaning.

+ Policies are **additive:** if two policies select the same Pod, the allowed set is the **union** — policies can only ever *add* permission, never subtract

Empty `podSelector: {}` = **every Pod in the namespace** (no `matchLabels` field at all)



Carry the four questions: *who's protected (podSelector)? who may call (from)? on what ports? and does a second policy union with this one?* — that's 90% of every NetworkPolicy review.

[↑ back to top](#)



Same grammar, opposite direction. And the industry-standard starting posture: **deny everything, then allow named flows**.

```
kind: NetworkPolicy
metadata: { name: default-deny-all, namespace: shop }
spec:
  podSelector: {}           # every pod in shop
  policyTypes: [Ingress, Egress]
  ingress: []               # empty = deny ALL inbound
  egress: []                # empty = deny ALL outbound
```



The classic default-deny self-own: deny-all **egress** instantly kills DNS — no pod can resolve *anything*. The very first egress allow in any cluster is DNS:

```
kind: NetworkPolicy
metadata: { name: allow-dns }
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels: { kubernetes.io/metadata.name: kube-system }
      ports:
        - { protocol: UDP, port: 53 }
        - { protocol: TCP, port: 53 }
```



The production playbook



CNI reality check: NetworkPolicy is just an API — **the CNI plugin must enforce it**. Calico, Cilium, Weave enforce; **flannel does not** (policies apply silently, block nothing). If policies "don't work", check your CNI before your YAML. Verify with Lab 3, not with hope.



One sentence: default-deny + explicit allows = the Kubernetes zero-trust baseline — and rule #1 of deny-all egress is **allow DNS first or nothing works and everything looks broken**.

↑ back to top



Book 01 asserted "every Service gets a DNS name." Here's the machinery: a **CoreDNS Deployment** in `kube-system`, fronted by the `kube-dns` Service — the cluster's phone book.

```
Pod resolv.conf → nameserver 10.96.0.10
  | query: "v1.default..."
  ▼
kube-dns Service (ClusterIP, stable – dogfooding!)
  ▼
CoreDNS Pods watch API Server → build zone in memory
  ▼
svc.cluster.local zone every Service & Pod IP, auto
```

```
# the cluster zone lives in a ConfigMap – the "Corefile":
$ kubectl -n kube-system get cm coredns -o yaml
Corefile: |
  .:53 {
    kubernet.es cluster.local # ← serves the cluster zone from the API
    forward . /etc/resolv.conf # everything else → upstream DNS
  }
```

Query form	Resolves	Works from
<code>v1</code>	same namespace	pods in <code>default</code> (search list fills the rest)
<code>v1.dev</code>	Service <code>v1</code> in ns <code>dev</code>	anywhere
<code>v1.dev.svc.cluster.local</code>	full FQDN	anywhere (always works)

🔧 Zero registration: "create a Service" is the DNS record — CoreDNS watches the API, no sidecar config

🔍 It also serves **Pod records** (`ip-10-244-1-5.default.pod.cluster.local`) and **SVC/AAAA** types — headless Services return Pod IPs directly



One sentence: CoreDNS is a small API-watching DNS server — the reason service discovery needs no setup, and the reason a broken `kube-dns` Service makes the entire cluster feel dead.

↑ back to top



dnsPolicy	Pod uses...	When
ClusterFirst default	CoreDNS (cluster names + forwarded externals)	99% of Pods
Default	the node's resolv.conf — no cluster names!	rare; e.g. heavy external-DNS workloads
ClusterFirstWithHostNet	CoreDNS even for hostNetwork Pods	hostNetwork daemons that still need Service names
None	whatever <code>dnsConfig</code> supplies	full manual control (custom nameservers/search)

🐞 ndots:5 — the sneaky perf tax

```
# a default pod's resolv.conf:
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
# any name with <5 dots is tried AS-IS *and* against every search
# suffix → "google.com" = 1 + 3 = 4 queries. measure, don't guess.
```

🔧 The debug ladder (Lab 4 runs it live)

```
$ kubectl exec dns -- nslookup v1           # 1 · short name (same ns)
$ kubectl exec dns -- nslookup v1.dev       # 2 · cross-namespace
$ kubectl exec dns -- nslookup kubernetes.default # 3 · the built-in canary
$ kubectl exec dns -- cat /etc/resolv.conf   # 4 · who's my resolver?
```

- ✅ FQDN resolves but short name fails → search list/ `ndots` issue, not DNS death
- ✅ nothing resolves + timeouts → check CoreDNS Pods (`-n kube-system`) and the `kube-dns` Service endpoints



One sentence: `ClusterFirst` quietly gives every Pod the cluster phone book plus upstream forwarding — and every "DNS is down" incident is solved by the four-command ladder above, in order.

↑ back to top



Ingress won the world but shows its age: routing power locked behind **vendor annotations**, cross-namespace routing unsafe, one blob of YAML mixing infra and app concerns. The **Gateway API** (SIG-Network, **GA since v1.0**) is the redesigned successor.

INGRESS (2015 MODEL)

behavior via per-controller **annotations**

cross-namespace = footgun territory

platform + app concerns in one object

not going away — huge installed base

GATEWAY API

native fields — headers, weights, method routing

cross-namespace via explicit [ReferenceGrant](#)

role-split objects (see below)

portable: any conforming implementation

```
# role separation – the headline idea:
GatewayClass # platform team: "we run nginx-gateway"
Gateway      # platform team: listener :443 + TLS cert
HTTPRoute    # APP team: my host/path → my svc ← attaches to the Gateway
# (+ GRPCRoute, TLSRoute... – and app teams never touch LB config)
```

```
kind: HTTPRoute
metadata: { name: shop }
spec:
  parentRefs: [{ name: edge-gateway }] # bind to the shared Gateway
  hostnames: [shop.example.com]
  rules:
    - matches: [{ path: { type: PathPrefix, value: /api } }]
      backendRefs: [{ name: api-svc, port: 80, weight: 90 }]
    - backendRefs: [{ name: web-svc, port: 80 }] # canary: weight 10
```



Adoption note: needs a Gateway-API controller installed (nginx-gateway, Envoy Gateway, Traefik v3+...). Guidance for new designs: *prototype on Ingress today, keep annotations minimal, know your escape hatch is Gateway API* — Book 08's GitOps tooling speaks both.



One sentence: Gateway API keeps Ingress's job but fixes its politics — **infrastructure (Gateway) owned by platform teams, routes (HTTPRoute) owned by app teams**, with portability and canary weights as first-class citizens.

[↑ back to top](#)



STEP 1 · Controller + two apps

```
$ minikube addons enable ingress
$ kubectl create deploy v1 --image=hashicorp/http-echo -- -text=hello-v1
$ kubectl create deploy v2 --image=hashicorp/http-echo -- -text=hello-v2
$ kubectl expose deploy v1 --port=5678 && kubectl expose deploy v2 --port=5678
# printf-style echo servers: perfect for proving WHO answered
```

STEP 2 · The rulebook

```
$ kubectl apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: routes }
spec:
  ingressClassName: nginx
  rules:
    - host: demo.local
      http:
        paths:
          - { path: /v1, pathType: Prefix,
              backend: { service: { name: v1, port: { number: 5678 } } } }
          - { path: /v2, pathType: Prefix,
              backend: { service: { name: v2, port: { number: 5678 } } } }
EOF
```

Add to `/etc/hosts` (admin): `<minikube ip> demo.local`

TEST

```
$ curl demo.local/v1
hello-v1
$ curl demo.local/v2
hello-v2 # path → service, proven
$ curl demo.local/v1/extra
hello-v1 # Prefix catches subpaths
```

PROOF IT'S L7

```
# same port 80 for both –
# only the PATH differed.
# A NodePort could never do
# this without 2 services
# on 2 ports.
```

Teardown: `kubectl delete ingress routes && kubectl delete deploy v1 v2 && kubectl delete svc v1 v2`

↑ back to top

**STEP 1 · Cert + Secret (Book 02 · CH 05 in action)**

```
$ openssl req -x509 -nodes -newkey rsa:2048 -days 365 \
  -keyout tls.key -out tls.crt -subj "/CN=demo.local"
$ kubectl create secret tls demo-tls --cert=tls.crt --key=tls.key
```

STEP 2 · Add the tls block to the Lab 1 Ingress

```
$ kubectl patch ingress routes --type=merge -p \
  '{"spec":{"tls":[{"hosts":["demo.local"],"secretName":"demo-tls"}]}}'
$ kubectl get ingress routes
HOSTS      PORTS      AGE
demo.local  80, 443    # 443 appeared
```

TEST HTTPS

```
$ curl -k https://demo.local/v1
hello-v1 # works! (-k: self-signed)
```

INSPECT THE CERT

```
$ curl -kv https://demo.local 2>&1 | grep subject
subject: CN=demo.local
# the EDGE serves your secret -
# pods never saw a cert
```

Proved: termination happens at the Ingress (CH 06) — one self-signed secret, HTTPS at the edge, plain HTTP behind.

Teardown: Lab 1 teardown + `kubectl delete secret demo-tls`.

[↑ back to top](#)



CNI check first: minikube default (bridge/kindnet) does **not** enforce policies. Restart with Calico, or run this lab on a cluster whose CNI enforces: `minikube delete && minikube start --cni=calico`.

STEP 1 · Players: a server, a labeled client, an unlabeled client

```
$ kubectl create deploy v1 --image=hashicorp/http-echo -- -text=hello-v1
$ kubectl expose deploy v1 --port=5678
$ kubectl run client --image=busybox:1.36 --restart=Never -- sleep 9999
$ kubectl run stranger --image=busybox:1.36 --restart=Never -- sleep 9999
```

STEP 2 · Deny all ingress to v1

```
$ kubectl apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: deny-v1 }
spec:
  podSelector: { matchLabels: { app: v1 } }
  policyTypes: [Ingress]
  ingress: []
EOF
$ kubectl exec client -- wget -qO- -T 3 http://v1:5678
wget: download timed out # ← BLOCKED. the room has a wall.
```

STEP 3 · Allow only labeled callers

```
$ kubectl apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-labeled }
spec:
  podSelector: { matchLabels: { app: v1 } }
  ingress:
    - from: [{ podSelector: { matchLabels: { access: "true" } } }]
EOF
$ kubectl label pod client access=true
$ kubectl exec client -- wget -qO- http://v1:5678 # now labeled
hello-v1 # ✅ allowed
$ kubectl exec stranger -- wget -qO- -T 3 http://v1:5678
wget: download timed out # ❌ still denied – labels are the keys
```

Proved: policy selects the server; pod labels select the callers; union of policies; enforcement is CNI-dependent. **Teardown:**

```
kubectl delete np deny-v1 allow-labeled && kubectl delete pod client stranger && kubectl delete deploy v1
&& kubectl delete svc v1
```



STEP 1 · The debug pod + the ladder

```
$ kubectl run dns --image=busybox:1.36 --restart=Never -- sleep 9999
$ kubectl exec dns -- nslookup kubernetes.default
Server:      10.96.0.10 # ← CoreDNS via kube-dns Service
Name:        kubernetes.default.svc.cluster.local
Address:     10.96.0.1 # ← the API Server's ClusterIP
```

NAME FORMS

```
$ kubectl exec dns -- nslookup kubernetes
# ✓ short name (same ns)
$ kubectl exec dns -- nslookup kube-dns.kube-system
# ✓ ns-qualified
$ kubectl exec dns -- nslookup \
  kube-dns.kube-system.svc.cluster.local
# ✓ full FQDN — always works
```

WHO CONFIGURED ME?

```
$ kubectl exec dns -- cat /etc/resolv.conf
nameserver 10.96.0.10
search default.svc.cluster.local
      svc.cluster.local cluster.local
options ndots:5
# dnsPolicy ClusterFirst wrote
# this file at pod start — CH 12
```

Proved: short names ride the search list; the resolver is the stable `kube-dns` VIP; `ndots:5` is why "external domains feel slow" (CH 12's perf tax). **Teardown:** `kubectl delete pod dns`.



Interview habit unlocked: when someone says "DNS is broken in the cluster," you now open with `kubectl exec ... nslookup kubernetes.default` — the canary that separates cluster-DNS death from an app's bad name.

↑ back to top



Q1. A LoadBalancer Service, examined closely, routes traffic to Pods via which underlying mechanism?
A. NodePorts on every node **B.** Direct Pod IP registration **C.** Ingress rules **D.** HostNetwork daemons

Q2. You apply an Ingress to a cluster with no controller installed. Result?
A. Apply fails with a validation error **B.** Ingress is created and works over NodePort fallback **C.** Ingress is created but silently does nothing — no ADDRESS ever appears **D.** The API Server starts a default controller

Q3. Which `pathType` matches `/api` and `/api/v1/users`?
A. Exact **B.** Prefix **C.** Whole **D.** Regex

Q4. An Ingress `tls` entry's `secretName` must reference a Secret of which type?
A. Opaque **B.** kubernetes.io/tls with `tls.crt` + `tls.key` **C.** dockerconfigjson **D.** basic-auth

Q5. NetworkPolicies apply but block nothing. Most likely cause?
A. Missing `ingressClassName` **B.** Policies need a namespace label **C.** The CNI (e.g. flannel) doesn't enforce NetworkPolicy **D.** `policyTypes` omitted

Q6. A NetworkPolicy has `podSelector: {}` and empty `ingress: []`. It affects...
A. every Pod in that namespace **B.** only unlabeled Pods **C.** the whole cluster **D.** nothing — empty selector is invalid

Q7. Two NetworkPolicies both select Pod X with different allow rules. X's effective access is...
A. the intersection (both must match) **B.** the last-applied policy only **C.** the first-created policy only **D.** the union — policies only ever add permission

Q8. The FQDN of Service `web` in namespace `prod` is...
A. `web.prod.local` **B.** `prod.web.svc.k8s` **C.** `web-svc.prod.default` **D.** `web.prod.svc.cluster.local`

Q9. With `dnsPolicy: Default`, a Pod resolves names using...
A. CoreDNS like everyone **B.** the node's own `resolv.conf` — cluster Service names fail **C.** hardcoded 8.8.8.8 **D.** the kubelet's cache

Q10. In Gateway API, which object carries the app team's routing rules?
A. Gateway **B.** GatewayClass **C.** HTTPRoute **D.** IngressClass

[↑ back to top](#)



Q	Ans	Why — one line
1	A	LB is NodePort + cloud wallet — the port column betrays it (CH 01–02)
2	C	Ingress = rulebook; no referee, no game, no error (CH 04)
3	B	Prefix matches the segment and everything under it (CH 05)
4	B	tls type mandates tls.crt/tls.key keys — Book 02 · CH 05 (CH 06)
5	C	Policy is an API; enforcement lives in the CNI — flannel won't (CH 10, Lab 3)
6	A	Empty selector = all Pods in the namespace; empty rules = deny (CH 09–10)
7	D	Additive union — policies grant, never revoke (CH 09)
8	D	<svc>.<ns>.svc.cluster.local — CH 11
9	B	Default = node's resolver; cluster names don't exist for that Pod (CH 12)
10	C	Gateway = shared listener; HTTPRoute = app-owned rules (CH 13)



9–10

Ready for Book 04



6–8

Re-read missed chapters' 
boxes

<6

Re-run Labs 1–4, then retake



Missed Q5 or Q7? Those are the two production incidents-in-waiting: unenforcing CNI (false security) and the additive-union rule (false restriction). Missed Q2? Remember: *apply succeeds* ≠ *traffic flows*.

[↑ back to top](#)



Exposition ladder — ClusterIP → NodePort → LoadBalancer; each rung delegates to the one below.

LoadBalancer — Service type; a cloud controller provisions an external LB fronting NodePorts. One per Service = one bill per Service.

MetalLB / minikube tunnel — ways to get LB behavior without a cloud (bare metal / local dev).

Ingress — L7 routing rulebook (host/path → Service); inert without a controller.

IngressClass — named controller type; `spec.ingressClassName` binds an Ingress to its referee.

Ingress Controller — running reverse proxy (ingress-nginx, Traefik...) that watches Ingress objects and renders live proxy config.

pathType — Prefix (segment + subtree) / Exact (literal) / ImplementationSpecific.

TLS termination — HTTPS ends at the Ingress; Pod traffic is plain HTTP unless re-encrypted deliberately.

Annotation — controller-specific behavior knob (rewrites, CORS...); powerful, non-portable.

NetworkPolicy — namespaced firewall rules: podSelector (target) + ingress/egress (from/to/ports); additive union; CNI-enforced.

Default-deny — posture where a selected Pod receives/sends nothing unless a policy allows it; empty rules array = deny all.

ipBlock / namespaceSelector / podSelector (in from/to) — the three source/destination selector types.

CoreDNS — cluster DNS Deployment in kube-system behind the kube-dns Service; serves svc.cluster.local by watching the API.

FQDN of a Service — `<name>.<namespace>.svc.cluster.local`; short names rely on the search list.

ndots:5 — resolv.conf option causing search-list expansion; the hidden cost behind "slow external names".

dnsPolicy — ClusterFirst (default) / Default (node's resolver) / ClusterFirstWithHostNet / None + dnsConfig.

Gateway API — GA successor to Ingress: GatewayClass + Gateway (platform-owned) and HTTPRoute (app-owned), portable and cross-namespacesafe via ReferenceGrant.

[↑ back to top](#)



TRAFFIC — COMMANDS

```
$ minikube addons enable ingress
$ kubectl get ingressclass
$ kubectl get ingress # ADDRESS must appear!
$ kubectl create secret tls t --cert=c --key=k
$ curl -k https://demo.local/api
# /etc/hosts: <minikube ip> demo.local
```

DNS — DEBUG LADDER

```
$ kubectl run dns --image=busybox:1.36 -it --rm \
  --restart=Never -- nslookup kubernetes.default
$ ... nslookup svc.ns.svc.cluster.local
$ kubectl exec dns -- cat /etc/resolv.conf
# kube-dns VIP = 10.96.0.10 by default
```

DEFAULT-DENY SKELETON

```
podSelector: {}
policyTypes: [Ingress, Egress]
ingress: [] # deny in
egress: [] # deny out
# FIRST allow: DNS udp+tcp 53 → kube-system
# policies are ADDITIVE (union)
# CNI must enforce (calico/cilium, NOT flannel)
```

LADDER + RULES OF THUMB

Need	Reach for
internal only	ClusterIP
quick demo, one app	NodePort
the one edge / TCP thing	LoadBalancer
HTTP host/path routing	Ingress
portable, canary, multi-team	Gateway API

FACTS WORTH A JOB OFFER

LB = NodePort underneath · Ingress without controller = silently dead (check ADDRESS + ingressclass)

TLS terminates at the edge; Pods stay HTTP unless deliberately re-encrypted

Annotations are vendor glue — Gateway API is the portable successor (GA v1.0)

NetworkPolicy: additive union · empty selector = whole namespace · empty rules = deny · AND within a from-item, OR across items

Deny-all egress kills DNS first — always allow :53 to kube-system

CoreDNS = API-watching DNS behind kube-dns VIP · ndots:5 = why external lookups multiply

[↑ back to top](#)



Book 01 built the machine, Book 02 dressed it — this book gave it **a front door, a phone book, and locks between the rooms.**

🌐 TRAFFIC — fully mapped:

└─ Ladder	ClusterIP → NodePort → LB (each rung delegates)
└─ Ingress	host/path rules → Services · TLS at the edge
└─ Controller	the proxy that enforces the rulebook
└─ NetworkPolicy	default-deny → explicit allows (DNS first!)
└─ CoreDNS	svc.cluster.local · ndots · dnsPolicy
└─ Gateway API	the portable future (Gateway + HTTPRoute)

✅ You can now...

Choose the right rung per Service — and explain the cost of picking wrong

Ship production Ingress: host/path rules, TLS secrets, controller triage (no ADDRESS? no controller)

Write a default-deny NetworkPolicy set that doesn't strangle DNS — and prove enforcement on the real CNI

Debug any "DNS broken" report with a four-command ladder instead of vibes

Place Gateway API in the roadmap and say precisely what it fixes about Ingress



Where to next — Book 04 · Scheduling & Node Management: your traffic arrives at the right door now — Book 04 decides **which machines run what**: the full taints/tolerations system, node & pod affinity, topology spread, Priority & Preemption, cordon/drain maintenance, and static Pods. You already tasted taints in Book 01's Lab 5 — now we master the whole discipline.

— End of Book 03 · 🎬 —